



BOOK ONE

Project Overview

What RuWork is, who uses it, what it can do, and how the pieces fit together

This document explains what RuWork is, who uses it, what it can do, and how the pieces fit together. Read it first. It gives you the mental model you need before the backend and frontend guides go into detail.

CONTENTS



1	What RuWork is	1
2	The three roles	2
	Student	2
	Job Provider	2
	Admin	3
3	What the system can do	4
	Accounts and access	4
	Jobs and work	4
	Feedback and communication	4
	Workspaces	5
	Production hardening	5
4	What RuWork deliberately does not do	6
5	Technology stack explained	7
	Backend	7
	Frontend	8
6	Architecture diagram	9
	The response path back	11
7	The email flow	12
8	Repository layout	13
9	How the project was built (phases)	14
10	Current verified status	15
11	Known discrepancies between the plan and the code	16
11.1	Application creation lives on the Job router, not the Application router	16
11.2	A real latent bug in scripts/seedDemo.js	16
	Where to go next	18

What RuWork is

RuWork is a **part-time job platform for University of Ruhuna students**. It connects two groups of people:

- **Students** who want flexible part-time work that fits around lectures.
- **Job Providers** (companies and individuals) who need short-term help.

A third group, **Admins**, keeps the platform trustworthy: they review who is allowed in, and they moderate content that breaks the rules.

The whole product is one idea repeated: *the server decides, not the browser*. A student can hide a button in their browser's developer tools, but they still cannot apply for a job they are not eligible for, because the server checks again every single time.

The three roles

ROLE	STORED IN COLLECTION	ROLE STRING IN CODE	HOW AN ACCOUNT IS CREATED
Student	users	student	Public self-registration, then email verification, then Admin approval
Job Provider	jobproviders	Job_Provider	Public self-registration, then email verification, then Admin approval
Admin	admins	admin	Privately provisioned only — <code>npm run create-admin</code> on the server

Why the odd `Job_Provider` spelling? *It was in the original supplied backend. Renaming it would have broken existing data and tokens for no real benefit, so it was deliberately preserved. You will see this exact string in `RuWork_backend-master/utils/account.js` → `JOB_PROVIDER_ROLE`.*

Student

A Student can browse jobs, apply, message the provider they are working with, track their applications, review a completed job, and manage their profile and password.

A Student only gets normal access if **all** of these are true (checked on the server, every request):

1. `role` is `student`
2. `email domain` is exactly `ruh.ac.lk`
3. `university` is exactly `University of Ruhuna`
4. `isEmailVerified` is `true`
5. `accountStatus` is `approved`
6. `moderationStatus` is `not suspended`

This is the “eligibility” rule, and it lives in `middlewares/authMiddleware.js` → `requireEligibleRuhunaStudent()`.

Job Provider

A Provider can post jobs, manage them, review applicants, accept or decline applications, mark work complete, message the student, and manage their company profile. Providers are **not** restricted to the university email domain — a real company will use its own domain.

Admin

An Admin approves or rejects registrations, suspends and restores accounts, hides and restores jobs and reviews, changes three platform-wide settings, and reads an immutable audit trail of every administrative decision.

Security reason: There is no public Admin sign-up route anywhere in the API. If there were, anyone who found it could grant themselves control of the platform. The only way to create the first Admin is to run a script on the server with environment variables that never leave that machine.

What the system can do

Accounts and access

- **Registration** — Students and Providers sign themselves up. The server forces `role`, `university`, `accountStatus`, and `isEmailVerified`, so a crafted request cannot make someone an approved Admin.
- **Email verification** — A one-time link proves the person controls the email address. Only a *hash* of the token is stored.
- **Admin approval** — A human reviews each registration. Verification and approval are two separate gates.
- **Authentication** — Login issues a signed JWT containing the account id, email, role, and a revocation counter.
- **Password change / reset** — Authenticated change (all roles) and emailed self-service reset (Students and Providers).
- **Token revocation and sign-out** — Changing a password or signing out invalidates every token already issued for that account.

Jobs and work

- **Job posting** — Providers create jobs as a `draft` or publish them `open`, with hourly or fixed pricing in LKR.
- **Job browsing** — Anyone can search, filter, sort, and page through open jobs. Hidden jobs, archived jobs, expired jobs, and jobs from suspended providers never appear.
- **Applications** — An eligible Student applies once per job with a 20–1000 character note.
- **Application lifecycle** — `pending_review` → `in_progress` / `declined` / `withdrawn`, and `in_progress` → `completed` / `cancelled`.
- **Agreed pricing** — The Provider can adjust the price when accepting. This is *information only*.

Feedback and communication

- **Reviews** — Only the Student, only for their own `completed` application, one active review each, rating 1–5.
- **Rating aggregates** — Each Job and each Provider stores a denormalised `averageRating` and `reviewCount`, recalculated whenever reviews change.

- **Messaging** — Direct Student↔Provider messages, scoped to one Application, with optional explicit contact-number sharing by the Student.
- **Notifications** — In-app notifications for the six application lifecycle events plus new messages.

Workspaces

- **Student dashboard**, applications, job history, profile.
- **Provider dashboard**, my jobs, applicants, company profile, reviews.
- **Admin dashboard**, registration reviews, student/provider administration, job moderation, review moderation, settings, audit trail.

Production hardening

- Security headers, a CORS allowlist, tiered rate limiting, bounded request bodies, centralized error handling, secret-redacting logs, a health endpoint, and validated fail-fast startup configuration.

What RuWork deliberately does not do

This matters as much as the feature list. If someone asks “where is the payment code?”, the answer is that there deliberately is none.

- **✗ No payment processing.** RuWork records prices, rates, budgets, and the provider-approved agreed price. It never collects, holds, transfers, or settles money, and it has no Paid/Pending status. Payment is arranged directly between the Student and the Provider. Both the Job Details and Application Details pages display this in `components/common/PaymentInformationCard.jsx`.
- **✗ No Admin access to private messages.** The Admin dashboard reports a *count* of messages, never their content, participants, or threads. There is no Admin messaging route at all.
- **✗ No destructive deletion of accounts or jobs.** Moderation is reversible. Suspending an account or hiding a job preserves every application, review, message, and audit record.
- **✗ No WebSockets or realtime push.** Unread counts refresh on navigation and explicit actions.
- **✗ No message attachments, editing, or deletion.**
- **✗ No refresh tokens or cookie sessions.** The bearer token plus server-side revocation is the entire session model.

Technology stack explained

Backend

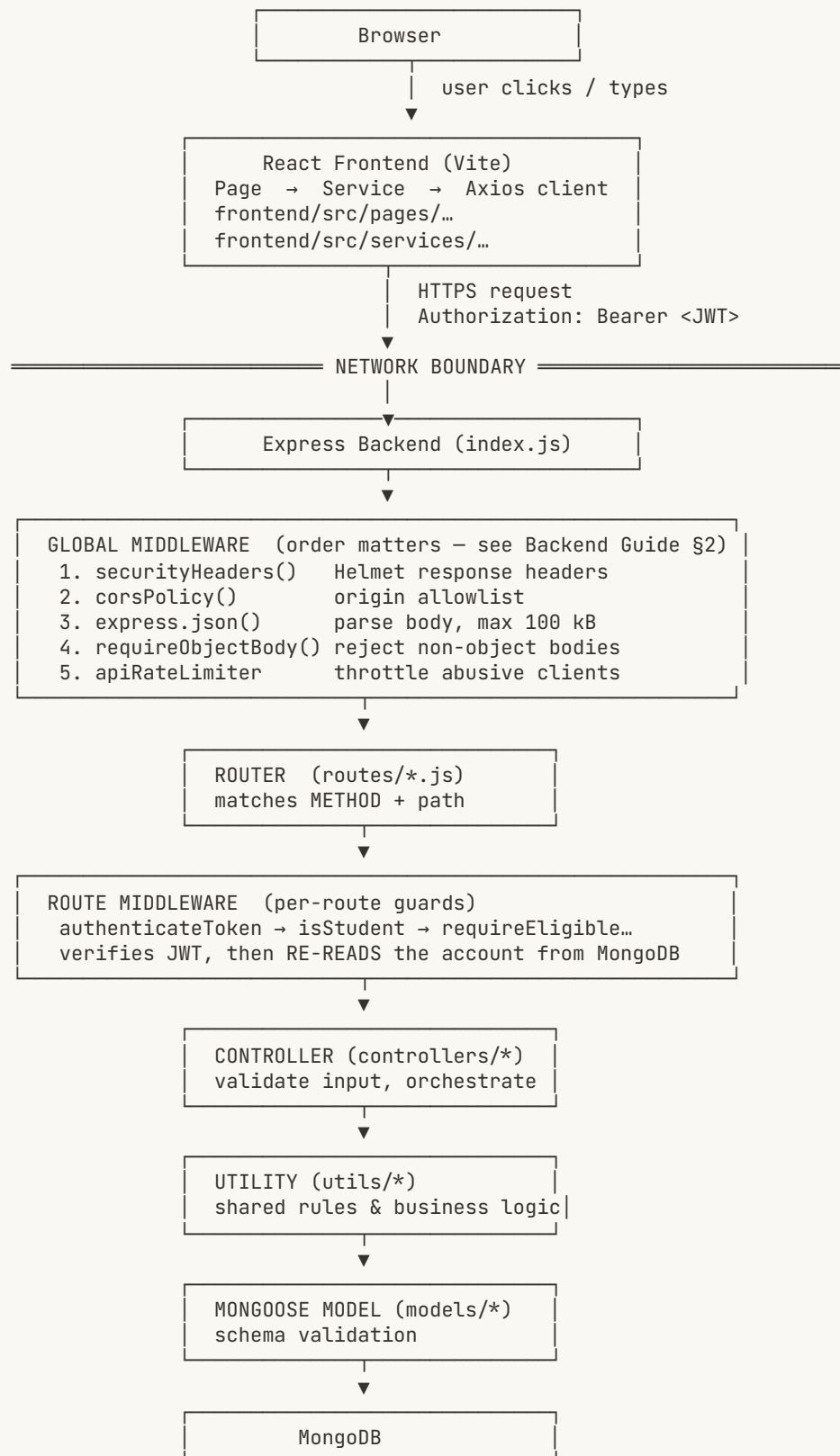
TECHNOLOGY	WHAT IT IS	WHY RUWORK USES IT
Node.js	A runtime that lets JavaScript run outside a browser, on a server.	Lets the team write the server in the same language as the frontend.
Express	A small web framework for Node. It turns incoming HTTP requests into <code>req/res</code> objects and lets you attach handlers to URLs.	Provides routing and the middleware pipeline that every RuWork request flows through.
MongoDB	A <i>document</i> database. Instead of tables and rows it stores JSON-like documents in collections.	Job postings and applications have varied, nested shapes; documents fit naturally.
Mongoose	A library that adds <i>schemas</i> on top of MongoDB — types, required fields, enums, validation, defaults, indexes.	MongoDB itself would accept literally any shape. Mongoose is what stops a job from being saved with a missing title or a negative price.
bcrypt	A deliberately slow password-hashing algorithm.	Turns a password into an irreversible hash so a database leak does not expose passwords.
jsonwebtoken (JWT)	Creates and verifies signed tokens.	Proves “this request comes from account X with role Y” without a server-side session store.
Nodemailer	Sends email over SMTP.	Delivers verification and password-reset links.
Helmet	Sets protective HTTP response headers.	Tells browsers to disable risky behaviours (framing, sniffing, referrer leakage).
cors	Implements Cross-Origin Resource Sharing.	Lets <i>only</i> the approved frontend origin call the API from a browser.
express-rate-limit	Counts requests per client and rejects excess.	Stops password-guessing and email-spam abuse.
dotenv	Loads <code>.env</code> into <code>process.env</code> .	Keeps secrets out of source code.

Frontend

TECHNOLOGY	WHAT IT IS	WHY RUWORK USES IT
React	A library for building UIs from reusable components whose output depends on <i>state</i> .	Job lists, forms, and dashboards all re-render automatically when data changes.
Vite	A fast dev server and production bundler.	Instant reloads while developing; an optimised, code-split bundle for production.
React Router	Client-side routing — swaps components based on the URL without a full page reload.	Gives RuWork its page structure and its role-protected route groups.
Axios	An HTTP client with interceptors.	One shared, configured client that attaches the JWT to every request.
Tailwind CSS	Utility-first CSS — you compose styles from small classes such as <code>flex</code> , <code>p-4</code> , <code>sm:grid-cols-2</code> .	Fast, consistent, mobile-first styling without a separate stylesheet per component.
Lucide React	An icon set as React components.	Consistent icons without image files.
Vitest	A test runner built for Vite.	Runs the frontend test suite.
React Testing Library	Tests components by querying them the way a <i>user</i> would (by label, by role).	Encourages testing behaviour rather than internal implementation.

Architecture diagram

The frontend and backend are two completely separate applications. They only ever communicate over HTTP/JSON.



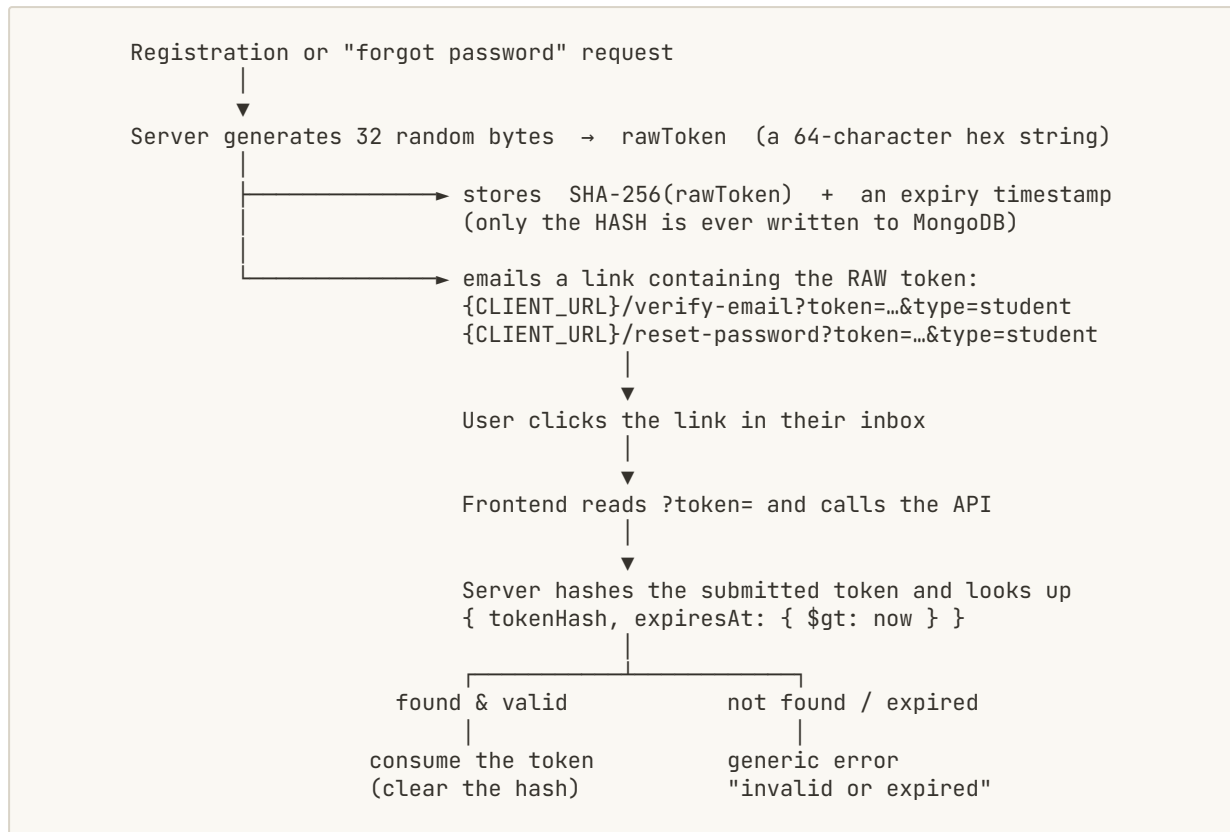
The response path back

```
MongoDB
  ▼ returns document(s)
Mongoose Model
  ▼ Mongoose document objects
Controller
  ▼ serializer picks ONLY safe fields (never password / token hashes)
  res.status(200).json({ ... })
  ▼
  errorHandler ← if anything threw, this converts it to a safe JSON error
  ▼ HTTP response
Axios response interceptor ← clears the session on a revoked token
  ▼
Service returns plain data
  ▼
Page setState(...) → React re-renders → user sees the result
```

The single most important idea in this diagram: the guard layer does **not** trust the JWT alone. It verifies the signature *and then reads the account out of MongoDB again* to check the account still exists, is still approved, and has not been suspended or had its tokens revoked. See Backend Guide §6.

The email flow

Email is a *side channel*. It proves the user controls an inbox, which is how RuWork verifies addresses and allows password resets.



Security reason: only the hash is stored. If someone steals a database dump, they get hashes, and a hash cannot be turned back into a working link. This mirrors exactly how passwords are handled — see `utils/emailVerification.js` and `utils/password.js`.

Sending is best-effort but honest. If SMTP fails during registration the account is still created, the verification token is rolled back so the user can immediately request a new one, and the API returns 503 with `VERIFICATION_EMAIL_NOT_SENT`. During a password reset a failed send rolls the token back but *still* returns the generic message, so nobody can use delivery behaviour to detect whether an address is registered.

Repository layout

```
YENU RUHUNA/
├── PROJECT_PLAN.md           ← the authoritative history/spec of all 10 phases
├── RuWork_backend-master/    ← the Express + MongoDB API
├── frontend/                 ← the React + Vite browser application
├── docs/
│   ├── RuWork_Backend_Requirements.pdf      (original supplied brief)
│   ├── RuWork_Backend_Objects_Relationships.pdf (original supplied brief)
│   └── developer-guide/                    ← you are here
└── design/                               ← supplied design references
```

The backend folder keeps its original name `RuWork_backend-master` and its original misspelled `middlewears/` directory. Both were preserved on purpose: renaming them would have created a large, risky diff with no functional benefit.

How the project was built (phases)

RuWork was built in ten deliberate phases. Each phase left the project in a working, tested state. `PROJECT_PLAN.md` records what each phase did and why.

PHASE	FOCUS
1	Backend foundation repair — removed hardcoded secrets, fixed role/email bugs, restored JWT/RBAC
2	Email verification, Admin approval APIs, eligibility middleware, private Admin provisioning
3	Frontend shell — Vite/React/Tailwind, landing, registration, login, verification, route protection
4	Job foundation — schema, pricing, ownership, browse/search/filter, provider job management
5	Applications — lifecycle, duplicate prevention, provider decisions, Option B job archiving
6	Role workspaces — live dashboards, profiles, job history, Admin registration reviews
7	Reviews and both rating aggregates
8	Messaging and notifications
9	Full Admin workspace — moderation, settings, audit trail, strict Admin authorization
10	Production hardening — security middleware, error handling, password lifecycle, token revocation, logging, health, code splitting

Current verified status

M easured on the current `master` commit (Phase 10 complete):

CHECK	RESULT
Backend tests	118 / 118 passing
Frontend tests	94 / 94 passing across 21 test files
Backend JavaScript files (<code>node --check</code>)	57 / 57 parse cleanly
Frontend ESLint	Passes, no errors or warnings
Vite production build	Succeeds; main chunk $\approx$ 377 kB ($\approx$ 119 kB gzipped)
<code>npm audit</code> (backend and frontend)	0 vulnerabilities each

Not verified, and not claimed: live MongoDB, live SMTP delivery, and real deployment. No external credentials or hosting environment are configured in this repository. Everything above was verified with automated tests and a temporary in-memory API.

Known discrepancies between the plan and the code

Documentation is only useful if it is honest. Two things in `PROJECT_PLAN.md` do not exactly match the code as it stands. **The code is the source of truth.**

11.1 Application creation lives on the Job router, not the Application router

`PROJECT_PLAN.md` §1 says “*Eligible-Student Application creation, listing/detail, withdrawal, and in-progress cancellation routes under `/api/applications`*”.

In the actual code, creation and the provider’s applicant list are mounted on the **Job** router:

OPERATION	ACTUAL ENDPOINT	DEFINED IN
Student applies to a job	POST <code>/api/jobs/:jobId/applications</code>	<code>routes/jobRouter.js</code>
Provider lists a job’s applicants	GET <code>/api/jobs/:jobId/applications</code>	<code>routes/jobRouter.js</code>
Student’s own applications	GET <code>/api/applications/my...</code>	<code>routes/applicationRouter.js</code>
Provider decisions	PATCH <code>/api/applications/provider/:id/...</code>	<code>routes/applicationRouter.js</code>

This is a reasonable design — an application is created *in the context of a job* — but the plan’s wording is imprecise. Use the table above.

11.2 A real latent bug in `scripts/seedDemo.js`

`scripts/seedDemo.js` creates Applications with a field named `studentNote`, but the Application schema in `models/application.js` requires `applicationNote` (required, 20–1000 characters).

```
// scripts/seedDemo.js — as currently written
await Application.create({
  "",
  studentNote: "I have prepared research datasets...", // ← wrong field name
  ""
});
```

Because the script has never been run against a live MongoDB (none is configured), Mongoose validation has never rejected it. **Running `npm run seed:demo` today would fail with a validation error saying `applicationNote` is required.**

This guide is documentation-only, so the bug has **not** been fixed here. It is a two-word change (`studentNote` → `applicationNote`, twice) and should be made — and then actually exercised against a real database — before anyone relies on the seed script.

Where to go next

- **Backend developer?** → *01_RuWork_Backend_Complete_Guide.md* — the most detailed document in this set.
- **Frontend developer?** → *02_RuWork_Frontend_Complete_Guide.md*
- **Confused by a keyword or symbol?** → *03_RuWork_Code_Glossary.md*
- **Want to trace one feature end to end?** → *04_RuWork_Request_and_Data_Flows.md*

